

Builder Design Pattern: AI Assistant

Course: Software Design Patterns

Assignment: Assignment #1 – Builder Pattern

Student: Melis Anel

Group: SE-2527

1. Introduction

The purpose of this project is to demonstrate the **Builder Design Pattern** using Java.

The selected topic is an **AI Assistant configuration**. An AI Assistant can contain several configurable properties, such as its name, model, language, response style, memory and specialization.

Creating such an object using one large constructor can make the code harder to read and maintain. The Builder Pattern allows the object to be created step by step and makes different configurations easier to manage.

This project contains two different AI Assistant configurations:

- Study AI Assistant
- Coding AI Assistant

The project demonstrates a Product, Builder interface, Abstract Builder, two Concrete Builders, a Director and a Client.

2. Why Builder Pattern?

The Builder Pattern is useful when an object has several configurable properties and can have different representations.

In this project, the `AiAssistant` object contains six fields:

```
private String name;  
private String model;  
private String language;  
private String responseStyle;  
private boolean memory;
```

```
private String specialization;
```

Using a constructor with all these parameters would make object creation less readable:

```
new AiAssIstant(  
    "Study ai",  
    "GPT-5",  
    "English",  
    "Simple",  
    true,  
    "Education"  
);
```

The Builder Pattern provides a more readable construction process:

```
new StudyAiassistantBuilderr()  
    .setName("Study ai")  
    .build();
```

The builder also allows different types of AI Assistants to be created with different default configurations.

3. Project Structure

The project contains the following main classes:

AiAssIstant

This is the **Product** class. It represents the final AI Assistant object.

It contains:

- name
- model
- language
- response style
- memory
- specialization

Aibuilderr

This is the **Builder interface**. It defines the methods required to configure and build an AI Assistant.

AbstractAiAssistantBuilder

This abstract class contains the common construction logic shared by the Concrete Builders.

It contains the common fields, setter methods and the `build()` method.

It also validates the required name field before creating the final object.

StudyAiassistantBuilderr

This is a Concrete Builder for a study-oriented assistant.

Its default configuration is:

- GPT-5
- English
- Simple response style
- Memory enabled
- Education specialization

CodingAiassistantbuilder

This is a Concrete Builder for a programming-oriented assistant.

Its default configuration is:

- GPT-5
- English
- Technical response style
- Memory disabled
- Programming specialization

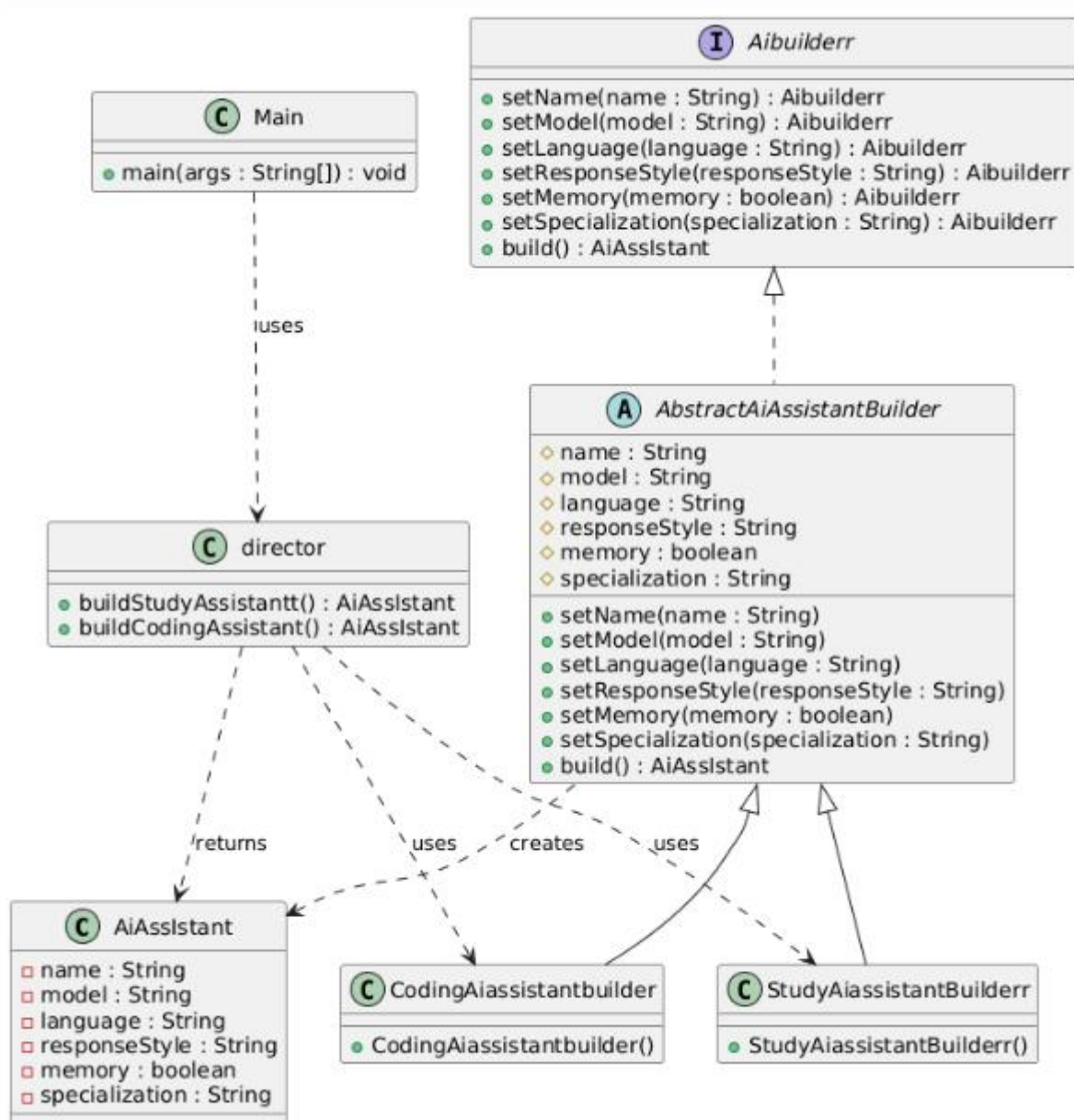
director

The Director provides predefined construction methods for the two assistant configurations.

Main

The Main class is the Client. It uses the Director to create the final objects and displays them.

4. UML Diagram



The UML diagram shows the structure of the Builder Pattern implementation. **AiAssistant** is the Product, **Aibuilderr** is the Builder interface, and the concrete builders provide different assistant configurations. The Director creates predefined Study and Coding AI Assistant configurations.

5. Implementation of the Builder Pattern

The Builder interface defines the construction steps:

```
public interface Aibuilderr {  
  
    Aibuilderr setName(String name);  
  
    Aibuilderr setModel(String model);  
  
    Aibuilderr setLanguage(String language);
```

```

    Aibuilderr setResponseStyle(String responseStyle);

    Aibuilderr setMemory(boolean memory);

    Aibuilderr setSpecialization(String specialization);

    AiAssIstant build();
}

```

The common implementation is placed in the abstract builder:

```

public abstract class AbstractAiAssistantBuilder implements Aibuilderr {

    protected String name;
    protected String model;
    protected String language;
    protected String responseStyle;
    protected boolean memory;
    protected String specialization;
}

```

The Study builder provides a study-oriented configuration:

```

public class StudyAiassistantBuilderr extends AbstractAiAssistantBuilder {

    public StudyAiassistantBuilderr() {
        setModel("GPT-5");
        setLanguage("English");
        setResponseStyle("Simple");
        setMemory(true);
        setSpecialization("Education");
    }
}

```

The Coding builder provides a different configuration:

```

public class CodingAiassistantbuilder extends AbstractAiAssistantBuilder {

    public CodingAiassistantbuilder() {
        setModel("GPT-5");
        setLanguage("English");
        setResponseStyle("Technical");
        setMemory(false);
        setSpecialization("Programming");
    }
}

```

```
}
```

The Director creates the predefined configurations:

```
public class director {

    public AiAssIstant buildStudyAssistantt() {
        return new StudyAiassistantBuilderr()
            .setName("Study ai")
            .build();
    }

    public AiAssIstant buildCodingAssistant() {
        return new CodingAiassistantbuilder()
            .setName("CodeHelper")
            .build();
    }
}
```

6. Fluent API

The project uses a **Fluent API**.

Each setter returns the current builder instead of returning void.

For example:

```
public AbstractAiAssistantBuilder setName(String name) {
    this.name = name;
    return this;
}
```

Because the method returns `this`, several methods can be chained:

```
new StudyAiassistantBuilderr()
    .setName("Study ai")
    .build();
```

This makes the construction process shorter and easier to read.

7. Clean Code Principles

7.1 Meaningful Names

Methods and fields use names that describe their purpose.

For example:

```
setResponseStyle()  
setSpecialization()  
buildCodingAssistant()
```

These names make the code easier to understand without additional comments.

The methods describe exactly what they change or create.

7.2 Single Responsibility Principle

Each class has a specific responsibility.

- `AiAssIstant` represents the final product.
- `Aibuilderr` defines construction steps.
- `AbstractAiAssistantBuilder` contains common building logic.
- `StudyAiassistantBuilderr` defines the study configuration.
- `CodingAiassistantbuilder` defines the coding configuration.
- `director` provides predefined construction sequences.
- `Main` runs the application.

This separation makes the project easier to understand and modify.

7.3 No Duplicated Construction Logic

Initially, both Concrete Builders contained the same fields, setter methods and `build()` method.

The common code was moved to:

`AbstractAiAssistantBuilder`

For example, the common `build()` method is implemented only once:

```
@Override  
public AiAssIstant build() {
```

```
if (name == null || name.isBlank()) {
    throw new IllegalStateException("Assistant name is required");
}

return new AiAssistant(
    name,
    model,
    language,
    responseStyle,
    memory,
    specialization
);
}
```

This avoids unnecessary duplication and makes future changes easier.

7.4 Small and Focused Methods

The setter methods perform only one operation.

For example:

```
public AbstractAiAssistantBuilder setModel(String model) {
    this.model = model;
    return this;
}
```

The method changes one field and returns the builder.

The methods are short and have a clear purpose.

7.5 Encapsulation

The Product class keeps its fields private:

```
private String name;
private String model;
private String language;
private String responseStyle;
private boolean memory;
private String specialization;
```

This prevents direct access to the internal state of the object from outside the class.

The object is created through the builder instead of allowing external code to freely modify its internal fields.

7.6 Validated Construction

The builder validates the required name field before creating the Product:

```
if (name == null || name.isBlank()) {  
    throw new IllegalStateException("Assistant name is required");  
}
```

This prevents an AI Assistant from being created without a required name.

Validation is performed inside `build()`, so the Product is created only after the required information has been checked.

8. Advantages of the Solution

The implemented Builder Pattern has several advantages.

Readable construction

The fluent API makes object creation easier to understand.

Different configurations

Different Concrete Builders can represent different types of AI Assistants.

Separation of responsibilities

The Product does not contain all construction logic.

Reduced duplication

Common builder logic is stored in the abstract builder.

Easy extension

A new type of AI Assistant can be added using another Concrete Builder without changing the Product class.

9. Disadvantages of the Solution

The Builder Pattern also introduces additional complexity.

First, the project requires several classes and interfaces. For a very simple object, this amount of structure would not be necessary.

Second, maintaining several builders can increase the amount of code when many product variations are introduced.

Therefore, the Builder Pattern is most useful when the object is sufficiently complex or has multiple possible configurations.

10. Conclusion

This project demonstrates how the **Builder Design Pattern** can be used to construct a complex AI Assistant object.

The solution includes a Product, Builder interface, Abstract Builder, two Concrete Builders, a Director and a Client.

The two Concrete Builders create different configurations for study and programming purposes. The Fluent API makes the construction process readable, while the Abstract Builder reduces duplicated construction logic.

The project also applies several Clean Code principles, including meaningful names, Single Responsibility, small methods, encapsulation, reduced duplication and validated construction.

Overall, the Builder Pattern provides a structured way to create different AI Assistant configurations while keeping the construction process separated from the final Product.

11. GitHub Repository

GitHub repository:

<https://github.com/anelmelis/aiassistant-builder-pattern.git>